open5Gcube: A Modular and Usable Framework for Mobile Network Laboratories

Thorsten Horstmann*§, Dominik Brunke‡, Tobias Kremeyer*§, Matthias Wilmes*, Gunnar Schneider*, Julian Sturm‡, Hartmut König‡, Michael Rademacher*§
*Fraunhofer FKIE, Bonn, Germany, firstname.lastname@fkie.fraunhofer.de
‡ZITiS, Munich, Germany firstname.lastname@zitis.bund.de
§Hochschule Bonn Rhein-Sieg, Sankt Augustin, Germany, firstname.lastname@h-brs.de

arXiv:2505.14501v1 [cs.NI] 20 May 2025

***Abstract*—In mobile network research, the integration of real-world components such as User Equipment (UE) with open-source network infrastructure is essential yet challenging. To address these issues, we introduce open5Gcube, a modular framework designed to integrate popular open-source mobile network projects into a unified management environment. Our publicly available framework allows researchers to flexibly combine different open-source implementations, including different versions, and simplifies experimental setups through containerization and lightweight orchestration. We demonstrate the practical usability of open5Gcube by evaluating its compatibility with various commercial off-the-shelf (COTS) smartphones and modems across multiple mobile generations (2G, 4G, and 5G). The results underline the versatility and reproducibility of our approach, significantly advancing the accessibility of rigorous experimentation in mobile network laboratories.**

***Index Terms*—5G, Testbed, Open-Source**

I. Introduction and Motivation

In mobile network research, a challenge is the integration of various Radio Access Network (RAN) and Core Network (CN) components for usage with real-world User Equipment (UE). This integration is needed to demonstrate ideas for future network generations or to uncover security vulnerabilities in existing standards. There are two worlds of mobile network components: Commercial solutions and open-source alternatives. Both come with advantages and disadvantages.

Commercial, closed-source systems are often expensive, and they do not provide the interfaces or modification capabilities required for in-depth experimental analysis. However, they (usually) offer a high stability and good compatibility. In contrast, open-source solutions offer flexibility and transparency that facilitate customization. However, these systems come with challenges, including a steep learning curve and potential compatibility issues with commercial off-the-shelf (COTS) UEs. This dichotomy highlights the need for a *modular and usable framework for mobile network laboratories* that combines the adaptability of Open source software (OSS) with the ease-of-use necessary for experimental setups.

From a security perspective, commercial systems frequently operate as black boxes, constraining access to internal mechanisms and impeding detailed vulnerability assessments. In contrast, open-source projects provide unrestricted access to source code, enabling researchers to scrutinize and modify implementations. This flexibility permits the development of robust proof-of-concept models for simulating attack vectors and assessing detection methods, thereby facilitating systematic evaluations and improvements of security measures within mobile communication frameworks. Such transparency is integral to advancing security protocols and ensuring resilience against emergent threat paradigms in 5G networks.

In this work, we introduce open5Gcube, a framework to enable researchers to conduct rigorous and reproducible experiments in mobile network research. Compared to previous work (see Section II), our framework 1) integrates the most common open-source mobile network projects into a unified management framework, 2) allows researchers to use these projects in different versions, and 3) was successfully used to test the compatibility of numerous COTS UEs.

II. Previous Work

Since testbeds play a crucial role in mobile network research, several publications have reported on their design and usage, each focusing on different aspects [1]–[14]. Multiple contributions provide detailed practical descriptions of the setup and configuration of a 5G Standalone (SA) network using OSS, highlighting central configuration parameters [8], [10], [12]. Other studies set up a 5G SA deployment and, using this deployment, perform measurements to evaluate the effect of different configuration parameters on relevant performance metrics, such as latency, throughput, and received signal power [4], [5], [7], [11]. A further significant body of research concentrates on comparative analyses of different 5G and 4G deployments, measuring the effect of different combinations of open-source Base Stations (BSs) and CNs on performance [2], [3], [6], [9]. Additionally, some studies implement advanced use cases for 5G networks using OSS projects, such as RANs based on Unmanned Aerial Vehicles [13] and network slicing aimed at very low latency services [14].

Table I provides a concise summary, highlighting software and hardware utilized across different publications. Please note that this list does not constitute an exhaustive survey. We present the ones we consider the most relevant and recent.

A mobile network laboratory, like real-world deployments, consists of three components: UEs, BSs, and the CN.

For **User Equipments**, previous studies identified three types: COTS smartphones; modems equipped with embedded

TABLE I: Overview of related work with a focus on used software and hardware.

Paper	Year	Unified Manag.	Config. available	UE			RAN						Core			
							Software				Hardware					
				Phones	Modem	Soft	srs	OAI	UERS.	Osmo	SDR	COTS	srs	Open5GS	Free5GC	OAI
[2]	2023	○	○	○	●	○	●	○	○	○	●	○	○	●	○	○
[3]	2024	○	○	○	●	○	●	●	○	○	●	○	○	●	○	●
[4], [5]	2024	◖	●	○	○	●	○	●	○	○	●	○	○	○	○	●
[6]	2023	○	○	●	●	●	●	●	○	○	●	●	●	●	○	●
[7]	2024	○	○	●	●	●	●	○	○	○	●	○	○	●	○	○
[8]	2022	◖	●	○	●	○	○	●	○	○	●	○	○	○	○	●
[9]	2023	◖	○	○	○	●	○	○	●	○	○	○	○	●	●	●
[10]	2024	◖	○	●	●	●	●	○	○	○	●	○	○	●	○	○
[11]	2023	○	○	○	●	○	○	●	○	○	●	○	○	○	○	●
[12]	2023	○	○	○	●	●	○	●	○	○	●	○	○	○	○	●
[13]	2024	○	○	●	●	●	●	○	○	○	●	○	○	●	○	○
[14]	2024	○	○	●	○	○	●	○	○	○	●	○	○	●	○	○
This	2025	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●

boards; and so-called Soft-UEs, which are purely software-based implementations running either on Software-Defined Radios (SDRs) or in emulated environments. As visualized in Table I, most previous work utilizes programmable modems due to their reported compatibility with various RAN and CN implementations. Smartphones and Soft-UEs are also commonly used. However, it has been noted that smartphone compatibility is not universal across open-source projects. It remains unclear whether incompatibilities primarily stem from the operating system (e.g., Android or iOS), baseband chipset (e.g., Exynos, Qualcomm, or MediaTek), or specific configuration parameters (e.g., frequency bands) [15]. Soft-UEs are useful for evaluating novel concepts such as new RAN-protocols; however, they lack real-world comparability [16].

Implementing a **Base Station** requires both hardware and software, typically OSS. Regarding hardware choices for base stations (BSs), most of previous work relies on SDRs, as indicated in Table I. Only a few projects incorporate commercial base stations (COTS BSs), likely due to higher costs and limited adaptability. Four OSS projects are predominantly utilized: srsRAN (srsRAN Project and srsRAN 4G) [17], OpenAirInterface (OAI) [18], UERANSIM [19], and OSMO-BTS [20]. Among these projects, srsRAN and OAI are most frequently employed within our evaluated subset of literature. OSMO-BTS is less commonly considered since it implements a 2G network; however, given well-known vulnerabilities and ongoing global availability of 2G networks, we include it [21].

A similar landscape of OSS exists for the **Core Network**. Implementations can be directly related to the BS software - such as srsRAN [17] or OAI [18] - or independent projects like Free5GC [22] or Open5GS [23], which focus on the CN.

In the context of this work, we emphasize two major aspects for **usability** and **reproducibility** of experimental testbeds:

- **Unified Management:** The presence of a framework facilitating orchestration across multiple projects to simplify the setup and maintenance (usability).
- **Configuration Availability:** Accessibility of project configurations provided as supplementary material alongside publications (reproducibility).

Surprisingly, both aspects remain largely unaddressed in the existing literature (see Table I). Only a few publications have made their configurations publicly accessible [4], [8]. Even less common is the implementation of unified management frameworks. Publications partially addressing this issue are discussed in the following. In [8], authors describe using Docker containers for automated setup and provide a detailed tutorial on configuring various components; however, their referenced repository is not publicly accessible. Mamushiane et al. [10] describe utilizing OpenStack with multiple CN implementations but provide limited details regarding their software infrastructure. The study by Reddy et al. [9] employs VirtualBox's graphical user interface as a basic management tool. Finally, Bozis et al. [4], [5] present an evolving laboratory setup at different development stages along with scripts to configure an OAI-based laboratory environment.

As summarized at the bottom of Table I and elaborated in subsequent sections, we contribute a framework implementing a unified management which we provide open-source alongside our configurations.

III. Architecture

The purpose of this section is to describe the design choices underlying our framework. We divided this section into two main parts: Hardware Design and Software Design.

A. Hardware Design

The primary objective of our hardware design is *flexibility*. Therefore, we divided the system into individual, interchangeable hardware components. Additionally, *mobility* is essential, as it allows the system to be easily transported to testing locations by fitting into a standard car. Another critical hardware requirement are *dedicated computing resources* in the radio domain to address high single-thread loads observed in individual processes. Thus, each computer requires a powerful CPU, which consequently generates significant heat. To ensure broad software compatibility, we selected the widely used Universal Software Radio Peripheral (USRP) X310 and USRP B210 SDRs from Ettus Research. The X310 SDR requires a 10G connection. Furthermore, precise synchronization among

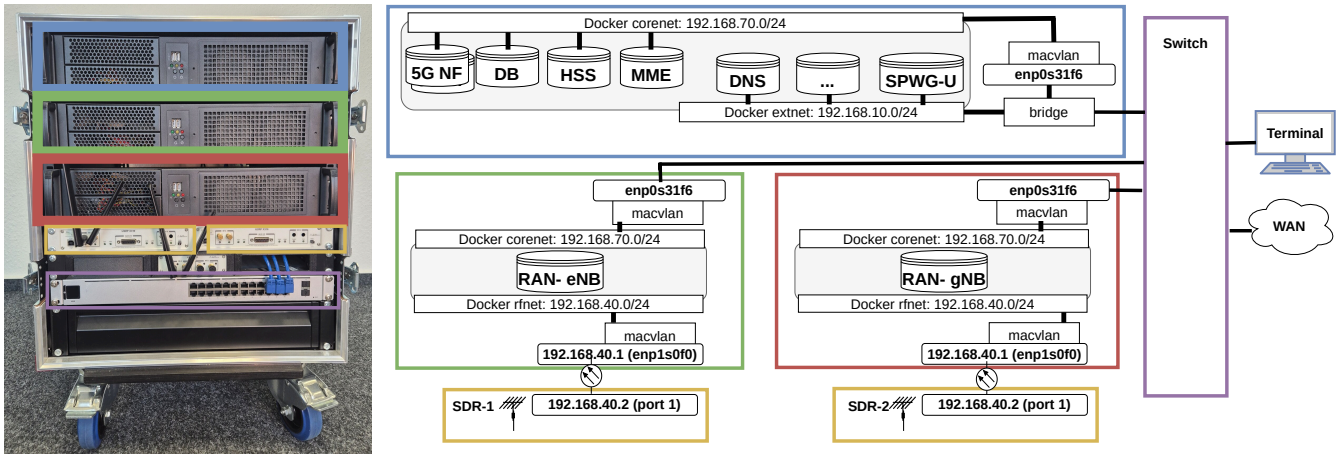

Fig. 1: Representation of our open5Gcube. On the left side, the components in the 19" rack. On the right side, the corresponding systems and their interconnections are depicted.

BSs in mobile networks is essential for communication between components; therefore, an external clock source is required. We transferred those requirements into a prototype which is shown in Fig. 1, and will be further detailed below.

Reference Hardware Setup: Our design includes three subunits: one CN and two BSs, supporting numerous typical scenarios, including a 5G Non-standalone (NSA) scenario with one eNodeB (eNB) and one gNodeB (gNB). Each subunit uses an identical computer system for self-containment and interchangeability. To facilitate mobility and be able to transport our system in a car, we use a mobile server rack in a 19-inch flight-case (max 12 Units (U)) with compact 2U servers featuring Intel i9 11900K CPUs and 10G network cards. A preconfigured 1U switch handles network connections.

The rack includes two X310 SDRs with Universal Baseboard Extension (UBX) daughterboards connected via 10G cables, a Quectel RM500Q-GL Development Kit, and optional space for B210 SDRs. A Leo Bodnar GPS disciplined oscillator (GPSDO) provides a stable 10 MHz reference signal for the SDRs, maintaining output even without a GPS fix [24].

Core System and Kernel: For maximal performance, hyper-threading, CPU frequency control, C-States, P-States and any other power management settings were disabled from the BIOS. Ubuntu 22.04 was selected as the operating system due to its compatibility with software package requirements of individual CNs. We installed and enabled a low-latency kernel (5.15.0-133) ensuring reliably low system response times essential for real-time performance in 5G applications. We observed that industrial products such as Amarisoft [25] employ similar operating systems and kernel configurations.

Network: The network topology consists of three interconnected systems communicating via Ethernet (1G) through the switch. Additionally, two RAN hosts establish direct 10G connections with one X310 SDR each.

As described in the following subsection III-B, services are deployed as Docker containers. Thus, the network architecture incorporates three virtual Docker networks:

- `corenet`: Configured as a macvlan [26] network, this network is designated for all the CN components. The use of 802.1q trunk bridge mode allows flexible distribution of containers across multiple servers. The host system is configured with additional virtual interfaces, facilitating direct communication with services within the network.
- `extnet`: Bridged to the WAN, this network facilitates the CN Gateway (SPGW-U, UPF) and other services (like DNS) for external connectivity through the host interface.
- `rfnet`: This network connects the USRP X310 SDRs, providing an isolated network for their operations.

B. *Software and Management*

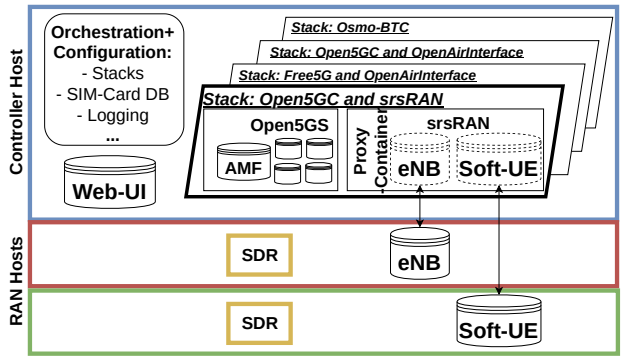

Fig. 2: Logical software organization of the framework. Open-source projects (e.g., Open5GS, srsRAN) are deployed by our framework on different host and summarized into *stacks*.

In this subsection, we describe the main design choices and functionality of our software framework, which runs on the reference setup described in the previous subsection. A visualization of the software framework is shown in Figure 2.

Containerization: Our framework utilizes container virtualization based on Docker. Using containers, services (e.g., eNB, gNB, 5G Network Functions (NFs)) run in isolated environments, providing benefits such as streamlined deployment, re-

duced dependency conflicts, and consistent installations across setups. Multiple services are grouped and organized into use-case-specific **stacks** (e.g., Open5GC-srsRAN), conveniently defined and managed using Docker-Compose files. Docker-Compose provides a declarative and straightforward mechanism to specify container configurations and interdependencies (RAN, CN NFs, databases, etc.), simplifying the setup and reproducibility of complex network scenarios involving different open-source projects.

Although the images are not optimized for size, they closely follow the structure of the official images associated with their respective projects as far as reasonable. All images are built upon a common base image. This approach ensures consistency and incorporates essential configurations. Additionally, the base image provides a user-friendly environment within a running container, facilitating convenient console interactions. It includes an up-to-date Wireshark version to easily trace and analyze network traffic inside the containers.

Orchestration: For orchestrating services across multiple hosts, two solutions were thoroughly evaluated: Kubernetes and Docker Swarm. Kubernetes, a widely recognized industry standard for container orchestration, inherently supports cloud-native setups and is adopted by Mobile Network Operators (MNOs) for 5G deployments due to its scalable architecture. Despite these advantages, Kubernetes exhibits substantial drawbacks for laboratory applications, including a steep learning curve and complex management largely unnecessary in smaller-scale laboratory settings. [27] In contrast, Docker Swarm—Docker's native orchestration solution—integrates seamlessly with Docker-Compose, providing straightforward development for laboratory-scale environments. [28]

However, a drawback shared by both Kubernetes and Docker Swarm is their conceptual overhead. From our experience, we found that neither system adequately represents our laboratory scenario. For instance, both solutions provide powerful scaling and load balancing mechanisms, which we do not require. On the other hand, with both solutions, assigning static fixed IP addresses to services is complex and needs extensive workarounds. This is because containers are considered as ephemeral and the networking is dynamically managed by these orchestration tools. Nevertheless, some of the open-source projects integrated in our framework require IP addresses rather than hostnames in their configuration files (e.g. OAI-RAN, Kamailio). Determining the IP addresses in such network setups at runtime would increase the integration effort significantly. We also consider a setup with fixed IP addresses easier to use and debug and especially more reproducible. In consequence, we argue that neither of both solutions is a good fit to our requirements.

Considering these trade-offs, we developed a **simplified orchestration mechanism**, tailored specifically for small-scale mobile network laboratory setups. Our approach employs a script-based implementation to minimize complexity. One host—termed the "Controller Host" (cf. Figure 2) centrally manages and initiates all containerized services. This host contains the project repository and operates all stack services (particularly those related to the CN), while delegating the execution of RAN containers to configured "RAN Hosts".

In more detail, the orchestration workflow functions as follows: Upon starting a stack on the Controller Host, RAN-related containers (such as eNB or gNB) are also initiated. The entry-point script of these containers includes logic to detect whether the current host matches the configured target host. Using Docker-outside-of-Docker (DooD), this script (executed by container entry-point) performs the following operations:

Require:
- Docker client is installed in the image
- Docker host socket is mounted into the container

Steps:
- Determine host machine name

if container is running on the configured RAN host **then**
  Start RAN component and **return**
end if
▷ *Release all assigned static IP addresses*
for all connected network interfaces **do**
  disconnect network from container
end for
- Connect the container to the default Docker bridge
- Transfer necessary files to the remote host via SSH
- Execute Docker-Compose on the remote daemon via SSH

This lightweight strategy leverages standard Docker-Compose features, extended by minimal scripting, and effectively simplifies remote deployment.

Networking: To effectively manage networking, Docker's built-in networking capabilities are utilized. The `corenet`, which is configured to use macvlan networking, allows containers to be attached directly to physical network interfaces with unique MAC addresses. This approach enables direct communication between services over the network without requiring additional routing or bridging setups. Consequently, services can easily be distributed across multiple physical hosts, significantly reducing complexity typically associated with multi-host network management.

Configuration: To facilitate the configuration of the stacks, we use simple central environment files which can be processed natively by Docker. Global settings such as channel number, frequency band, cell-Id etc. as well as the fixed IP addresses of the services are set centrally and used by the Docker-Compose files. We provide configuration files for all integrated projects where these parameters are defined as template variables. When the services are started, these settings are substituted in the configuration templates of the projects. Stacks can reuse the provided project configuration files and global settings or overwrite them if needed. The International Mobile Subscriber Identity (IMSI) and corresponding SIM card keys are specified in an environment file, too. On startup of a stack the particular subscriber database is populated from scratch to always have a clean state for the experiments.

Management: In addition to command-line-based interactions, we developed a simple Web-UI for interacting with the framework. Its principal use cases include configuring, starting and stopping specific stacks, and a live display of running containers and their logs, using color-coding to indicate the health status of the currently running stack.

The primary objective of our framework is to facilitate flexible combinations and rapid deployment of various open-source CN, RAN, and UE implementations. By providing a modular repository that includes prominent and commonly used open-source projects within the mobile communications community, different implementation combinations can be easily tested, as demonstrated in the following section.

IV. Evaluation

In this section, we present the results of experiments that can be conducted using the open5Gcube framework. Although our framework is application-driven and designed to realize specific mobile network scenarios (e.g., testing particular vulnerabilities, evaluating roaming scenarios, etc.), this section focuses primarily on results related to UE compatibility. To demonstrate the versatile nature of our framework, we evaluated all relevant mobile generations (2G, 4G, and 5G) using a fixed set of different UEs, as detailed in Table II.

We chose five different COTS smartphones from different vendors and a modem that has been commonly used in previous work (see Section II).

TABLE II: Overview of used UEs for testing the stacks.

Index	Model	OS Version	Modem
S25	Samsung Galaxy S25	Android 15	Snapdragon X80
OP13	OnePlus 13	Android 15	Snapdragon X80
ME50P	Motorola Edge 50 Pro	Android 14	Snapdragon X63
ARP6	Asus ROG Phone 6	Android 14	Snapdragon X65
GP9	Google Pixel 9	Android 15	Exynos 5400
AIP13	Apple iPhone 13 Mini	iOS 18.4	Snapdragon X60
QRM500	Quectel RM500Q-GL	ABR11A06M4G	Snapdragon X55

A. 2G

To build-up a 2G network, we defined a stack using the Osmocom implementation for cellular network infrastructure [20]. This stack includes the necessary subprojects to provide General Packet Radio Service (GPRS) for IP connections. Each component is encapsulated in a separated Docker container. The Base Transceiver Station (BTS) uses OsmoTRX connected to the USRP SDR as a back-end. We were able to connect all smartphones shown in Table II. Telephony, SMS, and the data packet service operated without any issues.

B. 4G + VoLTE

We evaluated our framework in a complex 4G scenario, which includes VoLTE and SMS services. The stack is based on the configuration described in [29] and illustrates the advantages of our modular approach, where the required projects can be flexibly combined. While the eNB is implemented by srsRAN, the CN with Home Subscriber Server (HSS) and Mobility Management Entity (MME) is integrated with Open5GS. The IP Multimedia Subsystem (IMS) is provided by the Kamailio SIP server project [30] and rtpengine [31]. The SMS service can be handled directly in the IMS or optionally provided via SMS-over-SGs by the Mobile Switching Center (MSC) of a parallel running 2G CN.

All tested UEs of Table II connected to the network and were able to establish a stable IP connection. In contrast, only three UEs connected successfully to the IMS. The problem of establishing a registration is covered by numerous issue entries in [29] and emphasizes the complexity of the VoLTE registration process [32]. Table III summarizes the results. Although VoLTE support is limited for various UEs, numerous experiments focus solely on a stable 4G data link.

TABLE III: UE Test results for 4G VoLTE stack.

	S25	OP13	ME50P	ARP6	GP9	AIP13	QRM500
4G data link	●	●	●	●	●	●	●
VoLTE registered	○	●	○	●	○	●	—

C. 5G SA

To explore the flexibility of our framework, we configured stacks to build 5G SA networks using open-source projects (see Table IV). Each CN implementation (Open5GS, OAI-Core, Free5GC) is combined with a RAN project (srsRAN, OAI-RAN) and tested separately with all UEs mentioned in Table II. In addition, each CN was connected to UERANSIM for an emulated setup. By using the global settings and configuration template files, as described in section III-B, it is not necessary to define any stack-specific configuration. So, a stack definition is simplified to just referencing the necessary services in a Docker-Compose file.

TABLE IV: Versions of the projects used in the stacks.

srsRAN	OAI-RAN	Open5GS	OAI-Core	Free5GC
24.10.1	2024-w23	2.7.2	2.1.0	3.4.4

Table V gives a summary of the test results. The Samsung Galaxy S25, OnePlus 13, Asus ROG Phone 6 and the Quectel RM500Q-GL worked with all stacks. Each UE connected to the RAN was authenticated by the CN and successfully established a Packet Data Unit (PDU) session. To stress test the link, a flood-ping was performed for two minutes without any connection loss.

The Motorola Edge 50 Pro connects to the RANs and establishes PDU sessions as well, but disconnects itself from the network after a few seconds due to the missing IMS. This is a known problem with some combinations of UE model and Android version [15] which could possibly be prevented by specific Carrier Setting configurations (e.g. with the OpenMobileNetworkToolkit [33]) or by disabling the feature with a hidden IMS settings menu.

In our tests, the Google Pixel 9 was unable to connect to srsRAN, as the cell itself was not detected by the device. With OAI-RAN, a link was established, but in combination with Free5GC no PDU session was created. With the two other CNs, the tests passed without any problems.

For the iPhone, the settings have been adjusted. In particular, the Subscription Permanent Identifier (SUPI) concealment was activated on the Sysmocom SIM card and configured

in Open5GS and Free5GC. In combination with srsRAN and OAI-RAN the iPhone connected to the private 5G network. Since the OAI-Core does not support SUCI-based authentication, the iPhone could not connect to it.

TABLE V: UE Test results for 5G SA stacks.

Stack		S25	OP13	ME50	ARP6	GP9	AIP13	QRM500
srsRAN	Open5GS	●	●	◖	●	○	●	●
	OAI-Core	●	●	◖	●	○	○	●
	Free5GC	●	●	◖	●	○	●	●
OAI-RAN	Open5GS	●	●	◖	●	●	●	●
	OAI-Core	●	●	◖	●	●	○	●
	Free5GC	●	●	◖	●	◖	●	●

In conclusion, the tests confirm that—from our perspective—the compatibility of the tested UEs is adequate for mobile network laboratories. During our experiments, we found that it was particularly helpful to switch quickly between stacks, to always have a consistent and reproducible setup and to simply change the versions of projects used.

V. Summary

Realistic testbeds are essential in mobile network research, particularly for security-related work. Commercial solutions lack the necessary flexibility while open-source alternatives tend to be complex to set up and maintain. To address this gap, **we developed open5Gcube, a modular framework that integrates prominent open-source projects into a unified management environment**. Leveraging Docker containerization, our architecture isolates services to simplify deployment and experimentation. Our prototype hardware setup consists of identical high-performance servers, SDRs and dedicated 10G networking infrastructure housed in a portable 19-inch rack optimized for transportability among laboratories. Extensive compatibility tests were conducted across multiple mobile generations (2G via Osmocom; 4G VoLTE via srsRAN/Open5GS; 5G SA using combinations of srsRAN, OAI-RAN, Free5GC, Open5GS) using various COTS smartphones and modems. The results demonstrate broad interoperability for basic data connectivity but selective compatibility for advanced features such as VoLTE registration. To encourage reproducibility and collaboration within the research community, **we have released open5Gcube as an open-source project** [34], allowing other groups to easily build their experiments and applications upon our framework.

References

[1] L. Schauer, T. Horstmann, S. Druesedow *et al.*, "Implementation of OpenAPI Wireshark Dissectors to Validate SBI Messages of 5G Core Networks," in *ITG-Fb. 316: Mobilkommunikation – Technologien und Anwendungen*. VDE Verlag, Offenbach, 2024.

[2] M. Amini, A. El-Ashmawy, C. Rosenberg *et al.*, "5G DIY: Impact of Different Elements on the Performance of an E2E 5G Standalone Testbed," in *GLOBECOM 2023 - 2023 IEEE Global Communications Conference*. IEEE, Dec. 2023, p. 6377–6382.

[3] M. Amini and C. Rosenberg, "A Comparative Analysis of Open-Source Software in an E2E 5G Standalone Platform," in *2024 IEEE Wireless Communications and Networking Conference*, apr 2024, pp. 1–6.

[4] E.-Z. G. Bozis, N. C. Sagias, M. C. Batistatos *et al.*, "A Versatile 5G Standalone Testbed Based On Commodity Hardware," in *2024 Panhellenic Conference on Electronics & Telecommunications (PACET)*. Thessaloniki, Greece: IEEE, mar 2024, pp. 1–4.

[5] ——, "Enhancing 5G Performance: A Standalone System Platform With Customizable Features," *AEU - International Journal of Electronics and Communications*, vol. 187, p. 155515, 2024.

[6] M. Chepkoech, N. Mombeshora, B. Malila *et al.*, "Evaluation of Open-Source Mobile Network Software Stacks: A Guide to Low-cost Deployment of 5G Testbeds," in *2023 18th Wireless On-Demand Network Systems and Services Conference (WONS)*. Madonna di Campiglio, Italy: IEEE, jan 2023, pp. 56–63.

[7] J. E. Håkegård, H. Lundkvist, A. Rauniyar *et al.*, "Performance Evaluation of an Open Source Implementation of a 5G Standalone Platform," *IEEE Access*, vol. 12, pp. 25 809–25 819, 2024.

[8] F. John, J. Schuljak, L. B. Vosteen *et al.*, "A Reference Deployment of a Minimal Open-Source Private Industry and Campus 5G Standalone (SA) System," in *2022 IEEE 10th International Conference on Information, Communication and Networks (ICICN)*. Zhangye, China: IEEE, Aug. 2022, pp. 1–9.

[9] R. Reddy, M. Gundall, C. Lipps *et al.*, "Open Source 5G Core Network Implementations: A Qualitative and Quantitative Analysis," in *2023 IEEE International Black Sea Conference on Communications and Networking (BlackSeaCom)*. IEEE, jul 2023, pp. 253–258.

[10] L. Mamushiane, A. Lysko, H. Kobo *et al.*, "Deploying a Stable 5G SA Testbed Using srsRAN and Open5GS: UE Integration and Troubleshooting Towards Network Slicing," in *2023 International Conference on Artificial Intelligence, Big Data, Computing and Data Communication Systems (icABCD)*. IEEE, aug 2023, pp. 1–10.

[11] A. Sahbafard, R. Schmidt, F. Kaltenberger *et al.*, "On the Performance of an Indoor Open-Source 5G Standalone Deployment," in *2023 IEEE Wireless Communications and Networking Conference*, mar 2023.

[12] M. Seidel, A. I. Grohmann, P. Sossalla *et al.*, "How to Get Away with OpenAirInterface: A practical Guide to 5G RAN Configuration," in *2023 3rd International Conference on Electrical, Computer, Communications and Mechatronics Engineering (ICECCME)*. IEEE, jul 2023, pp. 1–6.

[13] P. Valente, A. Teixeira, M. Luís *et al.*, "Easy to Deploy UAV-based Non-Public Open-Source 5G Network," in *2024 15th International Conference on Network of the Future*. IEEE, oct 2024, pp. 72–80.

[14] P. Vanichchanunt, O. Ritruechai, N. Wuttiananchai *et al.*, "Implementation of 5G Network Slicing Using Open Source Software," in *2024 12th Int. Electrical Engineering Congress*. IEEE, mar 2024, pp. 1–6.

[15] S. R. Systems, "COTS UEs - srsRAN Project documentation," https://docs.srsran.com/projects/project/en/latest/knowledge_base/source/cots_ues/source/index.html.

[16] M. Rouili, N. Saha, M. Golkarifard *et al.*, "Evaluating open-source 5g sa testbeds: Unveiling performance disparities in ran scenarios," in *NOMS 2024-2024 IEEE Network Operations and Management Symposium*. IEEE, May 2024, p. 1–6.

[17] Software Radio Systems, https://www.srsran.com/.

[18] OpenAirInterface Software Alliance, https://openairinterface.org/.

[19] Ali Güngär, https://github.com/aligungr/UERANSIM.

[20] https://github.com/osmocom/osmo-bts.

[21] G. Cattaneo, G. De Maio, P. Faruolo *et al.*, "A review of security attacks on the GSM standard," in *Information and Communication Technology-EurAsia Conference*. Springer, 2013, pp. 507–512.

[22] Linux Foundation, https://free5gc.org/.

[23] S. Lee, https://open5gs.org/.

[24] Leo Bodnar, "Precision GPS Reference Clock," https://www.leobodnar.com/.

[25] Amarisoft, https://www.amarisoft.com/test-and-measurement/device-testing/device-products.

[26] https://docs.docker.com/network/drivers/macvlan/.

[27] Linux Foundation, https://kubernetes.io/.

[28] Docker Inc, https://docs.docker.com/engine/swarm/.

[29] S. Herle, https://github.com/herlesupreeth/docker_open5gs.

[30] Kamailio SIP Server Project, https://www.kamailio.org/.

[31] Sipwise, https://github.com/sipwise/rtpengine.

[32] N. Jones, "Background to the 'VoLTE Mess'," dec 2024. [Online]. Available: https://nickvsnetworking.com/background-to-the-volte-mess/

[33] https://github.com/omnt/OpenMobileNetworkToolkit.

[34] https://github.com/open5Gcube/open5Gcube.